

Linear Regression & Non-Linear Curve Fitting

1 Observational Data and Experimental Variables

When analyzing physical systems, we gather experimental data to uncover underlying mathematical relationships between controlled physical inputs and measured laboratory outputs.

Definition 1.1 (Dataset, Features, and Targets). Suppose we record N distinct experimental observations. For each observation $i = 1, \dots, N$:

- We record m independent, controlled physical coordinates or state variables $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{im}) \in \mathbb{R}^m$, such as spatial position (x, y, z) , temperature T , pressure P , or time t . In machine learning, these controlled input coordinates are called **Features** or **Inputs**.
- We record a scalar physical response $y_i \in \mathbb{R}$, such as electric potential V , chemical concentration C , force F , or energy E . In machine learning, this observed detector output is called the **Target** or **Label**.
- The complete collection of N observational pairs $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ is called the **Training Dataset**.

2 The Physical Model and Parameter Space

We model our measured physical response y_i as a deterministic physical law $f(\mathbf{x}_i)$ perturbed by additive, random experimental noise ϵ_i :

$$y_i = f(\mathbf{x}_i) + \epsilon_i \quad (1)$$

Definition 2.1 (Model, Weights, and Bias). When we assume the physical response depends linearly on our m input coordinates, we write:

$$f(\mathbf{x}_i) = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_m x_{im} = \beta_0 + \sum_{j=1}^m \beta_j x_{ij} \quad (2)$$

- In machine learning, the functional form $f(\mathbf{x})$ used to approximate our physical system is called the **Model** or **Hypothesis**.
- The unknown physical invariants or coupling constants $\beta_1, \beta_2, \dots, \beta_m$ that scale each physical input variable are called **Weights** or **Parameters**.
- The constant term β_0 , which accounts for baseline reference values (such as ground potential V_0 or unstretched spring reference length L_0), is called the **Bias** or **Intercept**.

3 Residual Error and Objective Functions

Because every experimental detector has finite measurement precision, our predicted value $\hat{y}_i = f(\mathbf{x}_i)$ will not match the measured value y_i exactly. The difference $e_i = y_i - \hat{y}_i$ represents the observational residual.

Definition 3.1 (Loss Function and Optimization). To evaluate how poorly a given set of parameters fits our experimental data, we construct a single scalar error metric evaluated across the entire dataset.

- In machine learning, the scalar function evaluating prediction error across the dataset is called the **Loss Function** (or **Cost Function**).
- For standard linear regression, we define the Loss Function $\mathcal{L}(\beta)$ as the Mean Squared Error (MSE):

$$\mathcal{L}(\beta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \frac{1}{N} \sum_{i=1}^N (y_i - f(\mathbf{x}_i; \beta))^2 \quad (3)$$

- In physical chemistry and experimental physics, this is mathematically equivalent to minimizing the χ^2 error statistic. Assuming independent Gaussian detector noise $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$, minimizing MSE corresponds directly to Maximum Likelihood Estimation (MLE).
- The process of finding the optimal parameter vector $\hat{\beta}$ that minimizes $\mathcal{L}(\beta)$ is called **Training** or **Optimization**.

4 Catalog of Loss Functions in Scientific Machine Learning

Our choice of loss function $\mathcal{L}(\beta)$ specifies how we penalize discrepancies between model predictions and experimental measurements, encoding distinct statistical assumptions regarding detector noise and physical constraints.

4.1 Mean Squared Error (MSE / L_2 Loss)

$$\mathcal{L}_{\text{MSE}}(\beta) = \frac{1}{N} \sum_{i=1}^N (y_i - f(\mathbf{x}_i; \beta))^2 = \frac{1}{N} \|y - X\beta\|_2^2 \quad (4)$$

Physical Context: Represents MLE under independent Gaussian noise $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$. Smoothly differentiable everywhere, but sensitive to extreme measurement outliers due to squaring residual terms.

4.2 Mean Absolute Error (MAE / L_1 Loss)

$$\mathcal{L}_{\text{MAE}}(\beta) = \frac{1}{N} \sum_{i=1}^N |y_i - f(\mathbf{x}_i; \beta)| = \frac{1}{N} \|y - X\beta\|_1 \quad (5)$$

Physical Context: Represents MLE under Laplacian noise $p(\epsilon_i) \propto \exp(-|\epsilon_i|/b)$. Because residual errors are penalized linearly, MAE is robust against rare experimental glitches, sensor spikes, or extreme outliers.

4.3 Huber Loss (Smooth L_1 Loss)

$$\mathcal{L}_{\text{Huber}}(\beta) = \frac{1}{N} \sum_{i=1}^N e_i(\delta) \quad \text{where} \quad e_i(\delta) = \begin{cases} \frac{1}{2} (y_i - f(\mathbf{x}_i; \beta))^2, & \text{if } |y_i - f(\mathbf{x}_i; \beta)| \leq \delta \\ \delta \cdot |y_i - f(\mathbf{x}_i; \beta)| - \frac{1}{2} \delta^2, & \text{if } |y_i - f(\mathbf{x}_i; \beta)| > \delta \end{cases} \quad (6)$$

Physical Context: Acts as an L_2 loss within the expected noise tolerance $[-\delta, \delta]$ of the instrument, and transitions smoothly (C^1 continuous) to an L_1 penalty when residuals exceed threshold δ .

4.4 Regularized Loss Functions (Ridge and Lasso)

$$\mathcal{L}_{\text{Ridge}}(\boldsymbol{\beta}) = \frac{1}{N} \|y - X\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_2^2, \quad \mathcal{L}_{\text{Lasso}}(\boldsymbol{\beta}) = \frac{1}{N} \|y - X\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_1 \quad (7)$$

Physical Context: Appends parameter magnitude penalties controlled by hyperparameter $\lambda \geq 0$. Ridge (L_2) stabilizes ill-conditioned matrix inverses under co-linear inputs. Lasso (L_1) forces non-essential parameters to exact zeros, performing automated feature selection.

4.5 Physics-Informed Residual Loss (Differential Operator Penalty)

$$\mathcal{L}_{\text{PINN}}(\boldsymbol{\beta}) = \frac{1}{N} \sum_{i=1}^N (y_i - f(\mathbf{x}_i; \boldsymbol{\beta}))^2 + \lambda_{\text{phys}} \frac{1}{N_{\text{coll}}} \sum_{k=1}^{N_{\text{coll}}} \left(\mathcal{D} \left[f(\mathbf{x}_k^{\text{coll}}; \boldsymbol{\beta}) \right] - g(\mathbf{x}_k^{\text{coll}}) \right)^2 \quad (8)$$

Physical Context: Enforces governing differential equations $\mathcal{D}[f(\mathbf{x})] = g(\mathbf{x})$ (such as Poisson’s or Schrödinger’s equation) across N_{coll} unlabelled domain coordinates alongside experimental data points.

5 Fitting Non-Linear Curves with Trial Functions

Physical fields (such as electrostatic potentials, molecular orbitals, or interatomic potentials) are rarely straight lines across spatial coordinates $\mathbf{x}$.

Definition 5.1 (Trial Functions and Linear Fitting). A model is mathematically defined as **Linear** as long as it is linear *with respect to the unknown parameters* $\boldsymbol{\beta}$, regardless of how non-linear the chosen physical functions are with respect to the input coordinate $\mathbf{x}$.

To fit non-linear curves, we express $f(\mathbf{x})$ as a weighted sum of M chosen physical trial functions $\{\phi_j(\mathbf{x})\}_{j=1}^M$:

$$f(\mathbf{x}) = \sum_{j=1}^M \beta_j \phi_j(\mathbf{x}) = \beta_1 \phi_1(\mathbf{x}) + \beta_2 \phi_2(\mathbf{x}) + \cdots + \beta_M \phi_M(\mathbf{x}) \quad (9)$$

In machine learning, these trial functions $\phi_j(\mathbf{x})$ are called **Feature Functions**, **Feature Maps**, or **Transformations**.

Textbook Examples in Physics and Chemistry

1. **Quantum Chemistry (LCAO Method):** When modeling a molecular orbital wavefunction $\Psi(\mathbf{r})$, we build it as a sum of M atomic trial functions $\phi_j(\mathbf{r})$ (e.g., Gaussian functions):

$$\Psi(\mathbf{r}) = \sum_{j=1}^M c_j \phi_j(\mathbf{r}) \quad (10)$$

Finding the mixing coefficients $c_j = \beta_j$ from measured electron density values is a direct linear regression problem over the chosen atomic functions.

2. **Classical Electrodynamics (Multipole Expansions):** When measuring electrostatic potential $V(r, \theta)$ outside a charge distribution, we build it as a sum of radial trial functions:

$$V(r, \theta) = \sum_{l=0}^M B_l \left(\frac{P_l(\cos \theta)}{r^{l+1}} \right) \quad (11)$$

Here, our trial functions are $\phi_l(r, \theta) = \frac{P_l(\cos \theta)}{r^{l+1}}$, and parameters $\beta_l = B_l$ represent physical multipole moments (monopole B_0 , dipole B_1 , quadrupole B_2).

6 Model Capacity: Underfitting vs. Overfitting

When selecting the number of trial functions M (or polynomial degree) to fit physical observations, we encounter two distinct model failure modes.

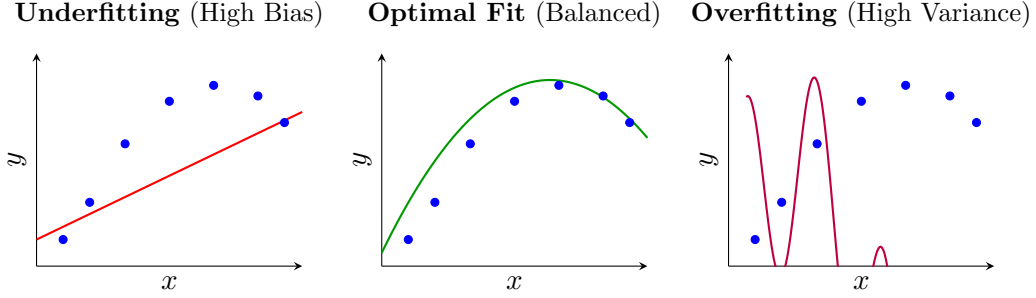


Figure 1: Model capacity regimes when fitting physical observational data.

Definition 6.1 (Underfitting (High Bias)). Occurs when a model lacks the functional capacity to capture the underlying physics (e.g., fitting a curved potential surface with a straight line).

Definition 6.2 (Overfitting (High Variance)). Occurs when a model has too many free parameters relative to sample size N . The model fits random, stochastic detector noise ϵ_i rather than true physical signal, causing wild non-physical oscillations between measurement coordinates.

7 The Bias-Variance Tradeoff

The Bias-Variance tradeoff represents a fundamental physical tension: how much we force our model to follow a rigid theoretical assumption versus how much we allow it to bend to fit noisy laboratory measurements.

7.1 Intuitive Physical Mechanism

When fitting physical fields $f(\mathbf{x}) = \sum_{j=1}^M \beta_j \phi_j(\mathbf{x})$, our primary capacity control knob is M (the number of trial functions):

- **Low Capacity (M Small):** High Bias, Low Variance. The model is too rigid to capture physical curvature, causing high systematic prediction error (Underfitting).
- **High Capacity (M Large):** Low Bias, High Variance. The model easily fits complex physical shapes, but fluctuates wildly across repeated noisy experimental trials (Overfitting).

7.2 Mathematical Derivation of Generalization Error

Suppose the true physical response at an unobserved position $\mathbf{x}_0$ is $y_0 = f(\mathbf{x}_0) + \epsilon$, with noise $E[\epsilon] = 0$ and $\text{Var}(\epsilon) = \sigma^2$. The expected squared prediction error of estimator $\hat{f}(\mathbf{x}_0)$ decomposes into three additive terms:

$$\begin{aligned}
\mathbb{E} \left[\left(y_0 - \hat{f}(\mathbf{x}_0) \right)^2 \right] &= \mathbb{E} \left[\left(f(\mathbf{x}_0) + \epsilon - \hat{f}(\mathbf{x}_0) \right)^2 \right] \\
&= \mathbb{E} \left[\left(\left[f(\mathbf{x}_0) - \hat{f}(\mathbf{x}_0) \right] + \epsilon \right)^2 \right] \\
&= \mathbb{E} \left[\left(f(\mathbf{x}_0) - \hat{f}(\mathbf{x}_0) \right)^2 \right] + 2\mathbb{E} \left[\left(f(\mathbf{x}_0) - \hat{f}(\mathbf{x}_0) \right) \epsilon \right] + \mathbb{E}[\epsilon^2] \quad (12)
\end{aligned}$$

Since measurement noise ϵ is uncorrelated with our model estimator $\hat{f}$, the expectation of the cross-term vanishes ($2\mathbb{E}[\dots]\mathbb{E}[\epsilon] = 0$). Adding and subtracting the expected model prediction $\mathbb{E}[\hat{f}(\mathbf{x}_0)]$ inside the first squared term:

$$\begin{aligned}
\mathbb{E} \left[\left(f(\mathbf{x}_0) - \hat{f}(\mathbf{x}_0) \right)^2 \right] &= \mathbb{E} \left[\left(\left[f(\mathbf{x}_0) - \mathbb{E}[\hat{f}(\mathbf{x}_0)] \right] + \left[\mathbb{E}[\hat{f}(\mathbf{x}_0)] - \hat{f}(\mathbf{x}_0) \right] \right)^2 \right] \\
&= \left(f(\mathbf{x}_0) - \mathbb{E}[\hat{f}(\mathbf{x}_0)] \right)^2 + \mathbb{E} \left[\left(\hat{f}(\mathbf{x}_0) - \mathbb{E}[\hat{f}(\mathbf{x}_0)] \right)^2 \right] \quad (13)
\end{aligned}$$

Substituting back yields the final error decomposition:

$$\begin{aligned}
\mathbb{E} \left[\left(y_0 - \hat{f}(\mathbf{x}_0) \right)^2 \right] &= \underbrace{\left(\mathbb{E}[\hat{f}(\mathbf{x}_0)] - f(\mathbf{x}_0) \right)^2}_{\text{Bias}^2 \text{ (Systematic Model Error)}} + \underbrace{\mathbb{E} \left[\left(\hat{f}(\mathbf{x}_0) - \mathbb{E}[\hat{f}(\mathbf{x}_0)] \right)^2 \right]}_{\text{Variance (Sensitivity to Sample Noise)}} + \underbrace{\sigma^2}_{\text{Irreducible Detector Noise}} \quad (14)
\end{aligned}$$

8 Practical Diagnostics: Train-Validation Splitting

To evaluate generalization capability in practice, we split our observational dataset $\mathcal{D}$ into two disjoint sets: a Training Set $\mathcal{D}_{\text{train}}$ (70%) to fit parameters $\hat{\beta}$, and a Validation Set $\mathcal{D}_{\text{val}}$ (30%) to evaluate prediction error on held-out data.

Regime	Train vs. Val Error	Diagnostic
Underfitting	$\mathcal{L}_{\text{train}} \approx \mathcal{L}_{\text{val}}$ (Both High)	High Bias
Optimal	$\mathcal{L}_{\text{train}} \approx \mathcal{L}_{\text{val}}$ (Both Low)	Balanced
Overfitting	$\mathcal{L}_{\text{train}} \ll \mathcal{L}_{\text{val}}$	High Variance

Table 1: Diagnostic indicators for model capacity regimes.

9 Matrix Formulations of Physical Systems

To process N experimental measurements across M trial functions simultaneously using linear algebra, we construct grid structures.

Definition 9.1 (Design Matrix and Overlap Matrix). We collect targets into vector $y \in \mathbb{R}^N$, parameters into vector $\beta \in \mathbb{R}^M$, and evaluate trial functions over experimental coordinates into the $N \times M$ **Design Matrix** X :

$$y = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}_{N \times 1}, \quad \beta = \begin{bmatrix} \beta_1 \\ \vdots \\ \beta_M \end{bmatrix}_{M \times 1}, \quad X = \begin{bmatrix} \phi_1(\mathbf{x}_1) & \phi_2(\mathbf{x}_1) & \dots & \phi_M(\mathbf{x}_1) \\ \phi_1(\mathbf{x}_2) & \phi_2(\mathbf{x}_2) & \dots & \phi_M(\mathbf{x}_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(\mathbf{x}_N) & \phi_2(\mathbf{x}_N) & \dots & \phi_M(\mathbf{x}_N) \end{bmatrix}_{N \times M} \quad (15)$$

The matrix product $X^T X \in \mathbb{R}^{M \times M}$, with elements $(X^T X)_{jk} = \sum_{i=1}^N \phi_j(\mathbf{x}_i) \phi_k(\mathbf{x}_i)$, represents the empirical **Gram Matrix** (or discrete **Overlap Matrix**) of our trial functions evaluated over our discrete sampling points.

10 Analytical Solution via Normal Equations

We write our total residual sum of squares $S(\beta)$ using matrix inner products:

$$S(\beta) = (y - X\beta)^T (y - X\beta) = y^T y - 2\beta^T X^T y + \beta^T X^T X \beta \quad (16)$$

Taking the gradient vector with respect to β and setting it to zero:

$$\nabla_{\beta} S(\beta) = -2X^T y + 2X^T X \beta = \mathbf{0} \implies X^T X \beta = X^T y \quad (\text{Normal Equations}) \quad (17)$$

Assuming X has full column rank ($\text{rank}(X) = M$), $X^T X$ is positive definite and invertible, providing the unique closed-form solution:

$$\hat{\beta} = (X^T X)^{-1} X^T y \quad (18)$$

11 Parameter Uncertainty and Propagation of Error

Extracted physical constants require variance bounds derived from experimental noise σ^2 . Applying the statistical covariance operator to our analytical estimator $\hat{\beta}$ under homoscedastic noise $\text{Cov}(y) = \sigma^2 I_N$:

$$\text{Cov}(\hat{\beta}) = \text{Cov}((X^T X)^{-1} X^T y) = (X^T X)^{-1} X^T (\sigma^2 I_N) X (X^T X)^{-1} = \sigma^2 (X^T X)^{-1} \quad (19)$$

Individual parameter variances are given by diagonal elements $\text{Var}(\hat{\beta}_j) = \sigma^2 [(X^T X)^{-1}]_{jj}$. When true noise variance σ^2 is unknown, we estimate it via residual variance s^2 :

$$s^2 = \frac{\|y - X\hat{\beta}\|_2^2}{N - M} \quad (20)$$

12 Executable Python Implementation

The following complete script demonstrates both practical diagnostic capacity sweeps (train-validation splitting) and physical parameter extraction with uncertainty propagation.

```
1 import numpy as np
2
3 def evaluate_capacity_sweep(x_data: np.ndarray, y_data: np.ndarray,
4                             train_fraction: float = 0.7):
5     """
6     Evaluates training and validation error across polynomial degrees
7     to practically identify underfitting and overfitting regimes.
8     """
9
10    N = len(x_data)
11
12    # Shuffle and split data into Training (70%) and Validation (30%)
13    indices = np.arange(N)
14    np.random.shuffle(indices)
15
16    split_idx = int(N * train_fraction)
17    train_idx, val_idx = indices[:split_idx], indices[split_idx:]
18
19    x_train, y_train = x_data[train_idx], y_data[train_idx]
20    x_val, y_val = x_data[val_idx], y_data[val_idx]
21
22    print(f"Capacity Sweep: {len(x_train)} Train, {len(x_val)} Val\n")
23    print(f"{'M':<3} | {'L_train':<10} | {'L_val':<10} | {'Status'}")
24    print("-" * 35)
25
26    for M in range(1, 8):
27        # Construct Design Matrix for Training Set using trial
28        # functions x^p
29        X_train = np.column_stack([x_train**p for p in range(M + 1)])
30
31        # Solve Normal Equations: beta = (X^T X)^(-1) X^T y
32        beta = np.linalg.pinv(X_train.T @ X_train) @ X_train.T @
33        y_train
34
35        # Compute Training Error (MSE)
36        L_train = np.mean((y_train - X_train @ beta) ** 2)
37
38        # Construct Design Matrix for Validation Set and Compute Error
39        X_val = np.column_stack([x_val**p for p in range(M + 1)])
40        L_val = np.mean((y_val - X_val @ beta) ** 2)
41
42        if M < 2:
43            status = "Underfitting"
44        elif L_val > 2.5 * L_train:
45            status = "Overfitting"
46        else:
47            status = "Optimal"
48
49        print(f"{'M':<3} | {'L_train':<10.4f} | {'L_val':<10.4f} | {status}")
50
51 def fit_electrostatic_potential(r_probes: np.ndarray, V_measured: np.
52                                 ndarray, max_moment: int = 2):
53     """
```

```

50     Fits multipole potential  $V(r) = \sum B_l / r^{(l+1)}$  and calculates
51     exact parameter uncertainties using  $\text{Cov}(\beta) = s^2 * (X^T X)^{-1}$ .
52     """
53     N = len(r_probes)
54     M = max_moment + 1
55
56     # Construct Design Matrix X (N x M) with trial functions  $\phi_l(r) = 1/r^{(l+1)}$ 
57     X = np.zeros((N, M))
58     for l in range(M):
59         X[:, l] = 1.0 / (r_probes ** (l + 1))
60
61     # Solve Normal Equations
62     XtX_inv = np.linalg.inv(X.T @ X)
63     beta = XtX_inv @ X.T @ V_measured
64
65     # Calculate Residual Variance ( $s^2$ )
66     residuals = V_measured - (X @ beta)
67     s_sq = np.sum(residuals**2) / (N - M)
68
69     # Covariance Matrix and Standard Errors
70     cov_beta = s_sq * XtX_inv
71     beta_errors = np.sqrt(np.diag(cov_beta))
72
73     return beta, beta_errors
74
75
76 if __name__ == "__main__":
77     np.random.seed(42)
78
79     # 1. Diagnostic Capacity Sweep
80     N_samples = 35
81     x_phys = np.linspace(0.5, 3.5, N_samples)
82     y_phys = np.sin(x_phys) + np.random.normal(0, 0.12, size=N_samples)
83     evaluate_capacity_sweep(x_phys, y_phys)
84
85     print("\n" + "="*40 + "\n")
86
87     # 2. Electrostatic Multipole Parameter Extraction
88     r_data = np.linspace(0.5, 4.0, 40)
89     V_data = 8.0 / r_data + 2.5 / (r_data**2) + np.random.normal(0,
90     0.10, size=40)
91
92     beta_hat, beta_err = fit_electrostatic_potential(r_data, V_data,
93     max_moment=2)
94     labels = ["Monopole (B0)", "Dipole (B1)", "Quadrupole (B2)"]
95
96     print("=== EXTRACTED PHYSICAL PARAMETERS ===")
97     for lbl, val, err in zip(labels, beta_hat, beta_err):
98         print(f"{lbl}: {val:.4f} +/- {err:.4f}")

```

Listing 1: Executable NumPy script for model diagnostics and physical parameter extraction.